

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ
УНИВЕРСИТЕТ «МИФИ»
(НИЯУ МИФИ)

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Методы оптимизации» на тему

**«ЧИСЛЕННЫЕ МЕТОДЫ ОДНОМЕРНОЙ
ОПТИМИЗАЦИИ»**

Выполнил: студент группы Б24-215

(подпись)

Гаганидзе Г. Г.

(Фамилия И.О.)

Проверил:

(оценка)

(подпись)

Климанов С. Г.

(Фамилия И.О.)

Москва 2026 г.

Оглавление

1.	Цель работы	2
2.	Теоретические сведения	2
3.	Описание реализации	4
3.1.	Общая структура программы	4
3.2.	Локальные методы	5
3.2.1	Пассивный поиск	5
3.2.2	Метод дихотомии	5
3.2.3	Методы Фибоначчи и золотого сечения	6
3.2.4	Метод парабол	8
3.3.	Глобальные методы	10
3.3.1	Равномерный перебор	10
3.3.2	Метод ломаных	10
3.4.	Визуализация и сравнение результатов	12
4.	Результаты вычислений	13
4.1.	Работа локальных методов	13
4.2.	Скорость сходимости локальных методов	17
4.3.	Глобальные методы	19
4.4.	Сравнение при одинаковом числе вычислений	19
5.	Вывод	22

1. Цель работы

Изучить и программно реализовать численные методы одномерной минимизации унимодальных и многоэкстремальных функций, провести сравнительный анализ их эффективности.

Среди реализованных методов для поиска локального минимума унимодальных функций были выбраны следующие:

- пассивный поиск;
- метод дихотомии;
- метод Фибоначчи;
- метод золотого сечения;
- метод парабол.

А также алгоритмы поиска глобального минимума многоэкстремальных функций:

- равномерный поиск;
- метод ломаных.

2. Теоретические сведения

Подробное обсуждение методов поиска минимума функции в этой работе опущено, но будут приведены основные определения и термины одномерной оптимизации, а также некоторые замечания по реализации алгоритмов.

Определение 2.1 (Унимодальная функция) Функция $f: X \rightarrow \mathbb{R}$ называется унимодальной на интервале X , если существует единственная точка минимума $x^* \in X$ и функция убывает слева от x^* и возрастает справа от него.

$$\exists x^* \in X : \begin{cases} f(x_1) > f(x_2), & \text{если } x_1 < x_2 < x^*, \quad \forall x_1, x_2 \in X; \\ f(x_1) < f(x_2), & \text{если } x^* < x_1 < x_2, \quad \forall x_1, x_2 \in X. \end{cases} \quad (1)$$

Все строго выпуклые функции являются унимодальными, но не все унимодальные функции являются выпуклыми. Некоторые разрывные функции могут обладать свойством унимодальности.

Функции многих экстремумов носят название мультимодальных (полимодальных). Реализация методов поиска минимума для мультимодальных функций требует дополнительных условий, например, знания числа экстремумов на интервале или константы Липшица для функции, что не всегда возможно на практике. Поэтому алгоритмы поиска глобального минимума требуют альтернативных подходов по сравнению с локальными методами.

Определение 2.2 (Липшицева функция) Функция $f(x)$ называется липшицевой на интервале $[a, b]$, если $\exists M > 0$ такая, что:

$$|f(x) - f(x')| \leq M|x - x'|, \quad \forall x, x' \in X \quad (2)$$

В силу того что, чтобы узнать константу Липшица M точно, необходимо решать задачу глобальной максимизации аналитической производной функции (либо давать ей грубую верхнюю оценку, что тоже исключает возможность автоматизации алгоритма), на практике используют приближенные значения константы Липшица. В работе реализован адаптивный способ уточнения константы Липшица по уже вычисленным значениям функции, таким образом снижается число вычислений функции и повышается точность оценки константы Липшица с каждой итерацией метода.

Также стоит упомянуть об одной особенности действия метода парабол, который использует аппроксимацию функции параболой через три точки, а именно поиск подходящей тройки точек для построения параболы. Было бы неэффективно, некорректно и ресурсоемко для каждой функции вручную подбирать точки исходя из ее графика, поэтому в работе реализован автоматический способ выбора тройки точек для построения параболы. Вычисления функции во время этого алгоритма также учтены в общем числе вычислений функции, что позволяет дать более объективную оценку эффективности сверхлинейного метода парабол.

Определение 2.3 (Отрезок локализации) Отрезком локализации называется отрезок $[a, b] \subset X$, содержащий точку минимума x^* :

$$x^* \in [a, b].$$

В процессе работы методов отрезок последовательно сужается, обеспечивая уточнение положения минимума. Бесконечношаговые алгоритмы должны иметь условия остановки, которые обеспечивают требуемую точность координаты минимума. На практике часто используются следующие условия остановки:

$$\|x^{k+1} - x^k\| \leq \varepsilon_1; \quad (3)$$

$$\|f(x^{k+1}) - f(x^k)\| \leq \varepsilon_2; \quad (4)$$

$$\|f'(x^{k+1})\| \leq \varepsilon_3. \quad (5)$$

Вообще говоря выполнение вышеуказанных условий не гарантирует нахождение глобального минимума, но в случае унимодальной функции, если алгоритм корректно реализован, выполнение этих условий гарантирует достижения необходимой точности решения задачи.

3. Описание реализации

Программа реализована на языке Python с использованием библиотек NumPy и Matplotlib. Реализация разделена на две группы: локальные методы для унимодальных функций и глобальные методы для функций с несколькими локальными экстремумами.

3.1. Общая структура программы

Для унификации результатов работы алгоритмов используется класс `OptimizationResult`. Он хранит не только найденную точку минимума, но и данные, необходимые для последующего анализа:

- найденный минимум;
- значение функции в найденной точке;
- количество вычислений функции;
- точки, в которых вычислялась функция;
- границы локализующего интервала;
- историю сходимости;
- историю наилучшего найденного значения для глобальных методов;
- текущую оценку константы Липшица.

Основные поля результата объединены в структуре данных:

```
1 @dataclass
2 class OptimizationResult:
3     x_min: float
4     f_min: float
5     n_evaluations: int
6     neighbors: np.ndarray
7     evaluation_points: np.ndarray
8     n_history: list[int] = field(default_factory=list)
9     length_history: list[float] = field(default_factory=list)
10    f_best_history: list[float] = field(default_factory=list)
11    x_best_history: list[float] = field(default_factory=list)
12    lipschitz_M: float | None = None
13
14    @property
15    def interval_length(self):
16        return self.neighbors[1] - self.neighbors[0]
```

Для каждого метода задаются точность по координате минимума `eps`, ограничение на число вычислений функции `i`, при необходимости, точность по значению функции `eps_f`.

3.2. Локальные методы

Локальные методы применяются к унимодальным функциям. Их общая идея состоит в последовательном уменьшении отрезка локализации. В программе реализованы пассивный поиск, методы дихотомии, Фибоначчи, золотого сечения и парабол.

3.2.1 Пассивный поиск

При пассивном поиске исходный интервал разбивается на равномерную сетку. Значение функции вычисляется в каждой внутренней точке, после чего выбирается точка с наименьшим значением функции. Соседние точки сетки образуют итоговый отрезок локализации.

Число точек выбирается из требования к погрешности координаты:

$$N \geq \frac{b - a}{2\varepsilon}. \quad (6)$$

3.2.2 Метод дихотомии

На каждой итерации метода дихотомии вычисляются значения функции в двух точках, расположенных симметрично относительно середины текущего интервала:

$$x_1 = \frac{a + b}{2} - \delta, \quad x_2 = \frac{a + b}{2} + \delta. \quad (7)$$

Если $f(x_1) < f(x_2)$, правая граница заменяется точкой x_2 ; иначе левая граница заменяется точкой x_1 . Процесс продолжается до достижения заданной длины интервала.

```
1  def dichotomy(function):
2      #...
3      while right - left > eps:
4          middle = 0.5 * (left + right)
5
6          x1, x2 = middle - delta, middle + delta
7          f1_value, f2_value = function(x1), function(x2)
8
9          if f1_value < f2_value:
10             right = x2
11         else:
12             left = x1
13
14     x_min = 0.5 * (left + right)
15     #...
```

3.2.3 Методы Фибоначчи и золотого сечения

Метод Фибоначчи выбирает внутренние точки в соответствии с отношением соседних чисел Фибоначчи. Перед началом итераций определяется такое число n , чтобы после заданного количества сужений длина интервала стала меньше требуемой точности. На каждой следующей итерации одна из уже вычисленных точек сохраняется, поэтому требуется только одно новое вычисление функции.

```
1  def fibonacci(function):
2      fib_need = (b - a) / (eps - delta)
3      n = fibonacci_numbers_until(fib_need)
4      # Выбираем в качестве [left, right] нормализованный интервал [0, 1]
5      x1 = left + (right - left) * fib(n - 2) / fib(n)
6      x2 = left + (right - left) * fib(n - 1) / fib(n)
7      #...
8      while n > 3:
9          if f1_value < f2_value:
10             right = x2
11             x2, f2_value = x1, f1_value
12             n -= 1
13             if n == 3: # F(1)/F(3) = F(2)/F(3) = 0.5
14                 break
15             x1 = left + (right - left) * fib(n - 2) / fib(n)
16             f1_value = normalized_function(x1)
17             n_evaluations += 1
18         else:
19             left = x1
20             x1, f1_value = x2, f2_value
21             n -= 1
22             if n == 3:
23                 break
24             x2 = left + (right - left) * fib(n - 1) / fib(n)
25             f2_value = normalized_function(x2)
26             n_evaluations += 1
27     x2 = x1 + normalized_delta
28     f2_value = normalized_function(x2)
29     if f1_value < f2_value:
30         right = x2
31     else:
32         left = x1
33     # Вычисление в исходных координатах
34     x_min = a + (b - a) * (left + right) / 2.0
35     #...
```

Метод золотого сечения использует постоянное отношение

$$\varphi = \frac{1 + \sqrt{5}}{2}.$$

В отличие от метода Фибоначчи, коэффициент сужения не меняется от итерации к итерации. Это позволяет применять один и тот же шаг до достижения требуемой точности. Этот метод можно считать частным случаем метода Фибоначчи при $n \rightarrow \infty$.

```
1  def golden_section(function):
2      #...
3      while True:
4          if f1_value < f2_value:
5              right = x2
6              x2, f2_value = x1, f1_value
7
8              if right - left <= normalized_eps:
9                  break
10             # Критерий остановки по числу вычислений
11             elif n_evaluations >= N:
12                 break
13
14             x1 = left + (right - left) / phi**2
15             f1_value = normalized_function(x1)
16             n_evaluations += 1
17
18         else:
19             left = x1
20             x1, f1_value = x2, f2_value
21
22             if right - left <= normalized_eps:
23                 break
24             elif n_evaluations >= N:
25                 break
26
27             x2 = left + (right - left) / phi
28             f2_value = normalized_function(x2)
29             n_evaluations += 1
30
31     x_min = a + (b - a) * (left + right) / 2.0
32     #...
```

3.2.4 Метод парабол

Метод парабол строит квадратичную аппроксимацию функции по трем точкам (x_0, f_0) , (x_1, f_1) и (x_2, f_2) , где $f_i = f(x_i)$. Центральная точка должна соответствовать меньшему значению функции:

$$x_0 < x_1 < x_2, \quad f_1 < f_0, \quad f_1 < f_2.$$

После чего в качестве новой точки рассматривает вершину построенной параболы. Для начального выбора тройки точек используется процедура схожая по принципу с методом дихотомии.

```
1  def bracketing(function, interval, N=None):
2      #...
3      while True:
4          #...
5          middle, f_middle = (left + right) / 2.0, function(middle)
6          n_evaluations += 1
7
8          if f_middle < f_left and f_middle < f_right:
9              # Возвращаем искомую тройку точек и значения функции в них,
10             # чтобы не пересчитывать повторно.
11             return (
12                 (left, middle, right),
13                 (f_left, f_middle, f_right),
14                 n_evaluations,
15                 #...
16             )
17         #...
18         if f_left < f_middle:
19             right = middle
20             f_right = f_middle
21         else:
22             left = middle
23             f_left = f_middle
```

Если параболическая аппроксимация вырождается или вершина находится вне текущего интервала, выполняется запасной шаг деления интервала. Это делает алгоритм устойчивее для функций, на которых построение параболы временно ненадежно.

```

1 def parabola(function, N=None, show=True):
2     #...
3     bracket, values, n_evaluations, evaluation_points, boundary_interval =
4     ↪ bracketing(function, (a, b), N=N)
5     (x0, x1, x2) = bracket
6     (y0, y1, y2) = values
7
8     while x2 - x0 > eps:
9         extremum = parabola_vertex((x0, x1, x2), (y0, y1, y2))
10        use_parabola = extremum is not None
11
12        if use_parabola and (not x0 < extremum < x2 or np.isclose(extremum,
13        ↪ x1)):
14            use_parabola = False
15
16        if not use_parabola: # Запасной шаг деления интервала
17            if x1 - x0 > x2 - x1:
18                extremum = 0.5 * (x0 + x1)
19            else:
20                extremum = 0.5 * (x1 + x2)
21
22        y_extremum = function(extremum)
23        n_evaluations += 1
24
25        if extremum < x1:
26            if y_extremum < y1:
27                x2, y2 = x1, y1
28                x1, y1 = extremum, y_extremum
29            else:
30                x0, y0 = extremum, y_extremum
31        else:
32            if y_extremum < y1:
33                x0, y0 = x1, y1
34                x1, y1 = extremum, y_extremum
35            else:
36                x2, y2 = extremum, y_extremum
37
38        n_history.append(n_evaluations)
39        length_history.append(x2 - x0)
40
41        x_min = 0.5 * (x0 + x2)
42    #...

```

3.3. Глобальные методы

Для мультимодальных функций в программе реализованы равномерный перебор и метод ломаных. В отличие от локальных методов, они не предполагают, что на всем исходном интервале существует только один минимум.

3.3.1 Равномерный перебор

Метод перебора строит равномерную сетку из N точек и вычисляет функцию в каждой из них. Найденная точка является приближением глобального минимума, а половина длины шага сетки задает оценку погрешности по координате:

$$\|x_{\min} - x^*\| \leq \frac{b - a}{2N}.$$

Для анализа процесса поиска в программе сохраняются последовательности лучших найденных значений x_k и f_k после каждого вычисления функции.

3.3.2 Метод ломаных

Реализация этого метода в работе основана на адаптивном подборе константы Липшица: на каждой итерации ее значение обновляется по максимальному коэффициенту наклона между соседними вычисленными точками.

$$M_{\text{new}} = C_{\text{safety}} \max_i \left| \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} \right|. \quad (8)$$

Множитель C_{safety} ¹ используется как запас относительно наблюдаемого максимального наклона. Без этого запаса нижние ломаные могли бы проходить через уже вычисленные точки, что снижало бы эффективность локализации минимума.

```
1  def update_lipschitz(x_points, y_points, current_M=-np.inf, safety=1.09,):
2      order = np.argsort(x_points)
3      x_sorted = np.array(x_points)[order]
4      y_sorted = np.array(y_points)[order]
5
6      slopes = np.abs(np.diff(y_sorted) / np.diff(x_sorted))
7      new_estimate = safety * np.max(slopes)
8
9      return max(current_M, new_estimate)
```

¹Значение множителя $C_{\text{safety}} = 1.09$ подобрано экспериментально.

```

1 def broken_lines(function, N=None, show=True):
2     #...
3     current_M = max(update_lipschitz(x_points, y_values), 1e-6)
4     while True:
5         R_values = []
6         t_values = []
7
8         for j in range(len(x_points) - 1):
9             #... вычисление координат точки пересечения ломаных
10            t = 0.5 * (x_left + x_right) - (y_right - y_left) / (2.0 *
11                ↪ current_M)
12            R = 0.5 * (y_left + y_right) - current_M * (x_right - x_left) / 2.0
13
14            R_values.append(R)
15            t_values.append(t)
16
17            # Наименьшее значение R соответствует наиболее перспективной точке
18            # для вычисления функции
19            best_index = np.argmin(R_values)
20            lower_bound = R_values[best_index]
21
22            # Нестрогий критерий остановки
23            if np.min(y_values) - lower_bound < eps_f:
24                break
25            if n_evaluations >= max_evaluations:
26                break
27
28            x_new = t_values[best_index]
29            y_new = function(x_new)
30
31            if y_new <= min(y_values):
32                x_best_history.append(x_new)
33            else:
34                x_best_history.append(x_best_history[-1])
35
36            insert_index = np.searchsorted(x_points, x_new)
37            x_points = np.insert(x_points, insert_index, x_new)
38            y_values = np.insert(y_values, insert_index, y_new)
39            current_M = update_lipschitz(x_points, y_values, current_M)
40
41            index_min = np.argmin(y_values)
42            x_min = x_points[index_min]

```

3.4. Визуализация и сравнение результатов

Функция `visualize` строит два графика. На первом показывается функция на всем исходном интервале, точки вычислений и найденный минимум. На втором отображается увеличенная окрестность итогового отрезка локализации. Графики сохраняются в каталоге `results/<method>/` с именем тестовой функции.

Для исследования сходимости сохраняются число вычислений функции и длина текущего интервала локализации. Эти данные используются для построения графика в логарифмическом масштабе. Отдельно реализовано сравнение методов при одинаковом числе вычислений функции, что позволяет сравнивать не только итоговую точность, но и вычислительную эффективность алгоритмов.

4. Результаты вычислений

Вычислительные эксперименты проведены для трех тестовых функций:

$$\begin{aligned}f_1(x) &= (x - 2)^2 + 1, & x &\in [0, 5]; \\f_2(x) &= x^2 + 2 \sin(3x), & x &\in [0.8, 3]; \\f_3(x) &= x^2 + 5 \sin(2x) + 2 \cos(3x), & x &\in [-3, 3].\end{aligned}$$

Унимодальные функции f_1 и f_2 использовались для проверки локальных методов, а многоэкстремальная f_3 — для сравнения глобальных методов. В качестве критериев оценки рассматривались найденная координата минимума, длина итогового отрезка локализации, число вычислений функции и динамика сходимости.

4.1. Работа локальных методов

Для каждой локальной процедуры на рисунке показаны график функции на всем интервале, точки вычислений, найденный минимум и увеличенная окрестность итогового отрезка локализации.

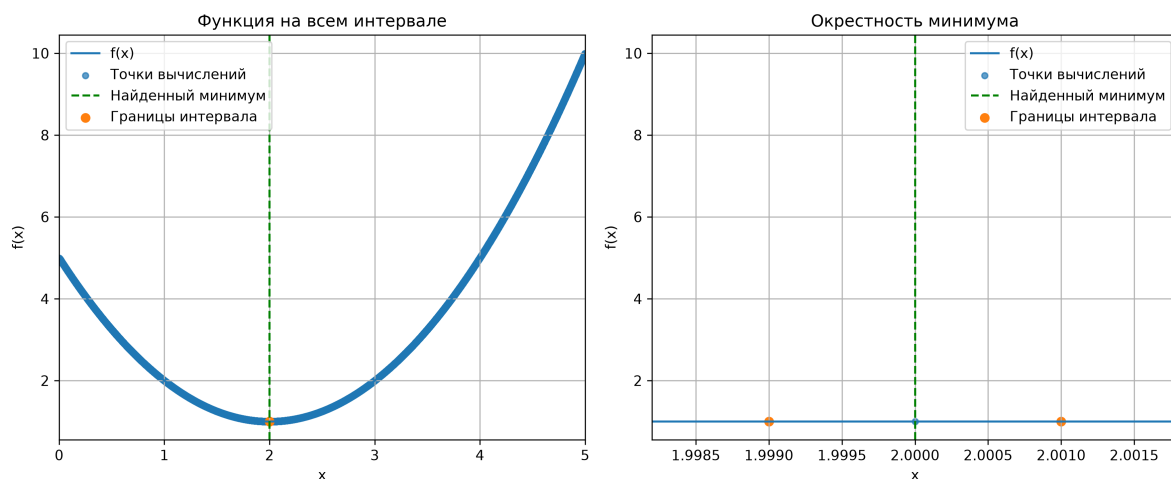


Рис. 1. Пассивный поиск на функции f_1 .

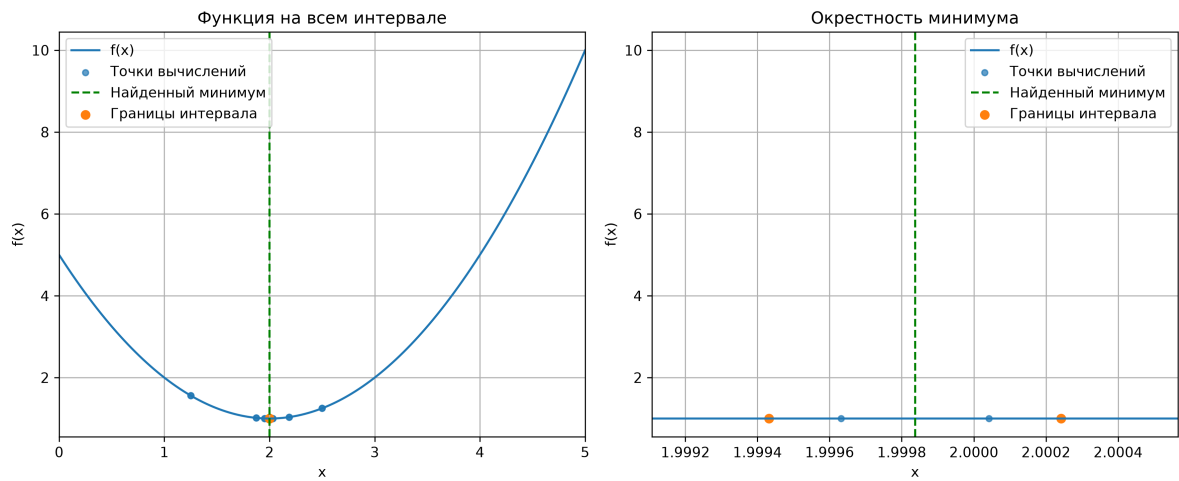


Рис. 2. Метод дихотомии на функции f_1 .

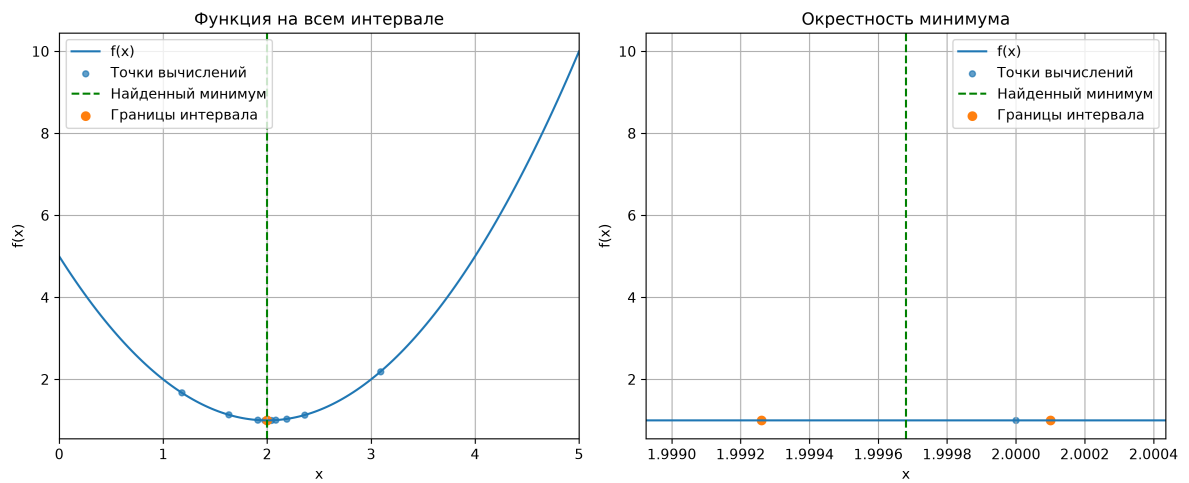


Рис. 3. Метод Фибоначчи на функции f_1 .

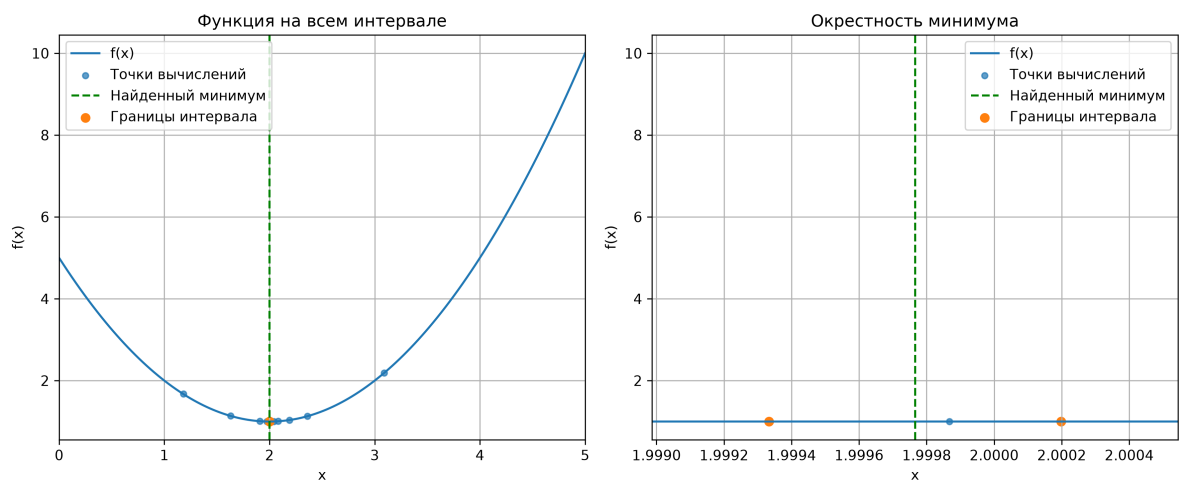


Рис. 4. Метод золотого сечения на функции f_1 .

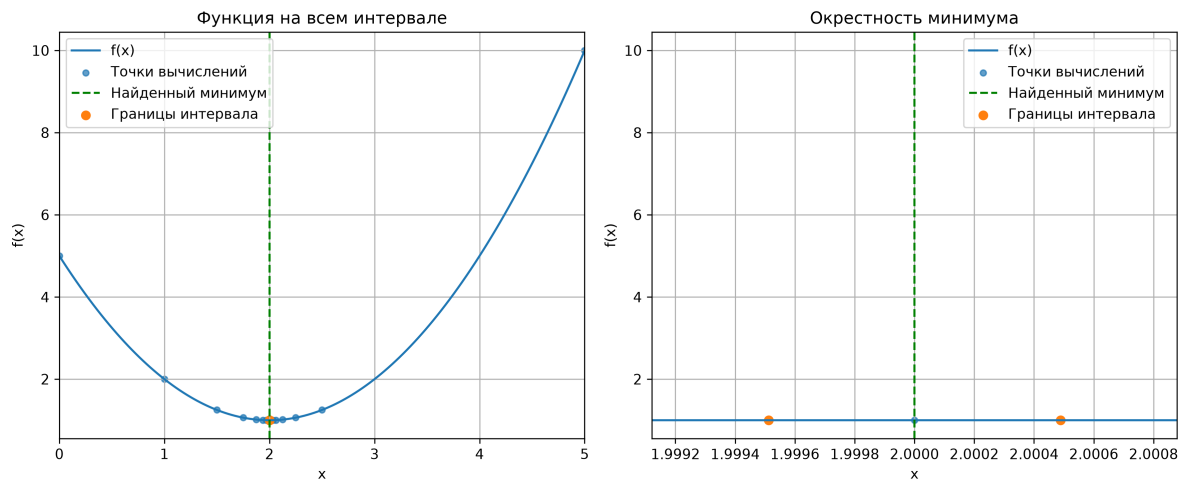


Рис. 5. Метод парабол на функции f_1 .

На квадратичной функции f_1 все методы локализации демонстрируют ожидаемое поведение: вычисления концентрируются около единственного минимума $x = 2$. Методы дихотомии, Фибоначчи и золотого сечения последовательно сужают интервал, тогда как пассивный поиск равномерно исследует весь отрезок.

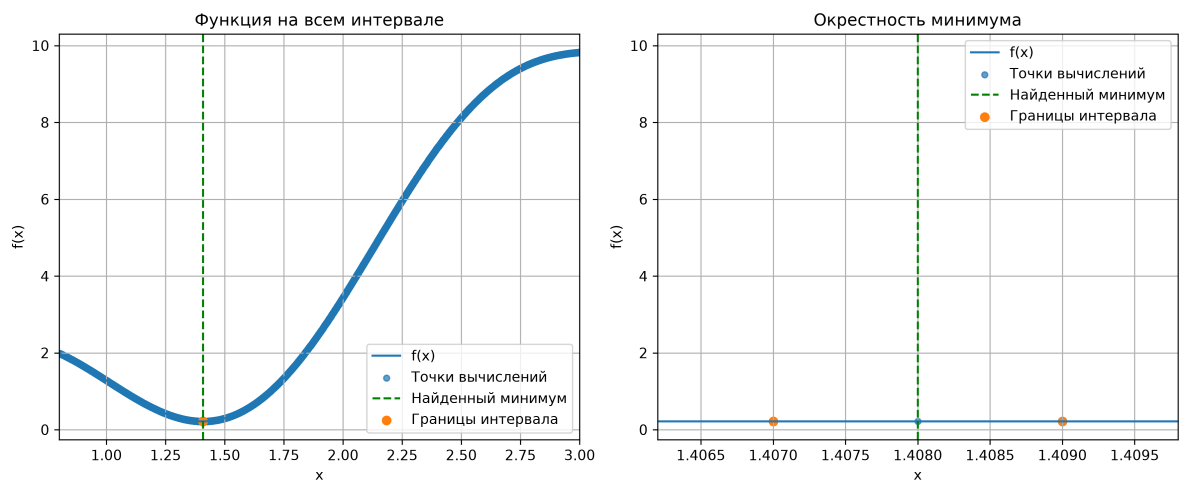


Рис. 6. Пассивный поиск на функции f_2 .

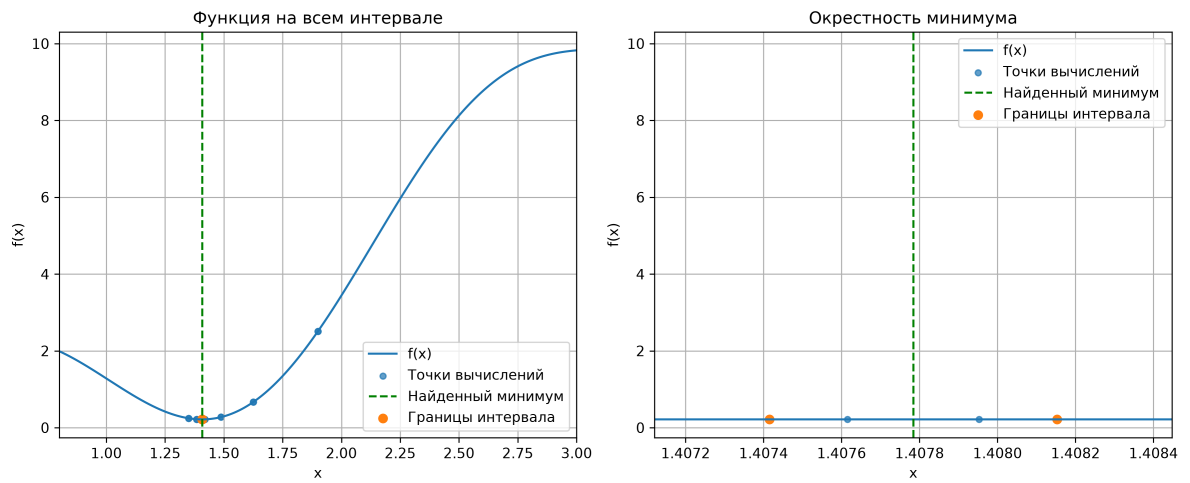


Рис. 7. Метод дихотомии на функции f_2 .

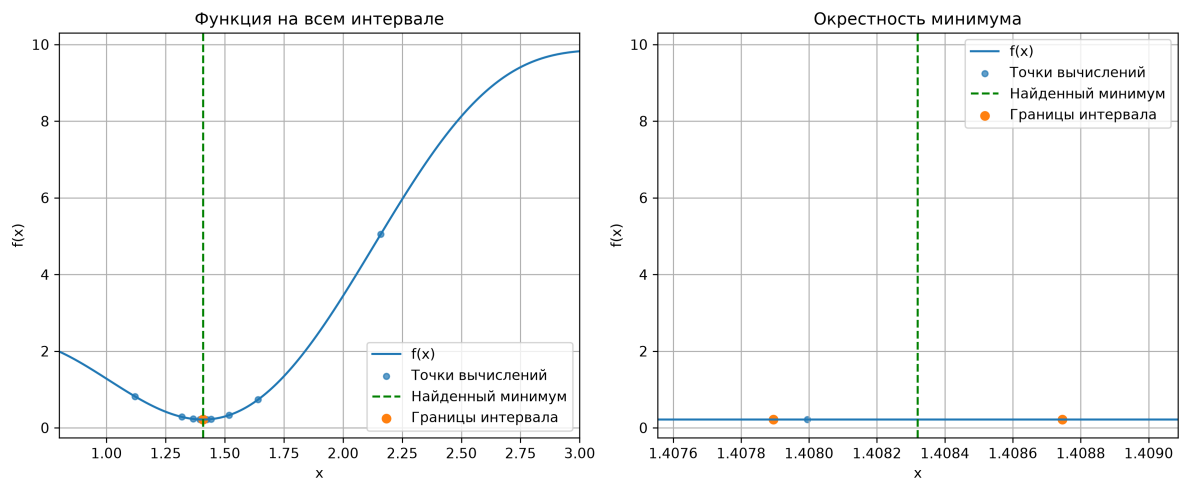


Рис. 8. Метод Фибоначчи на функции f_2 .

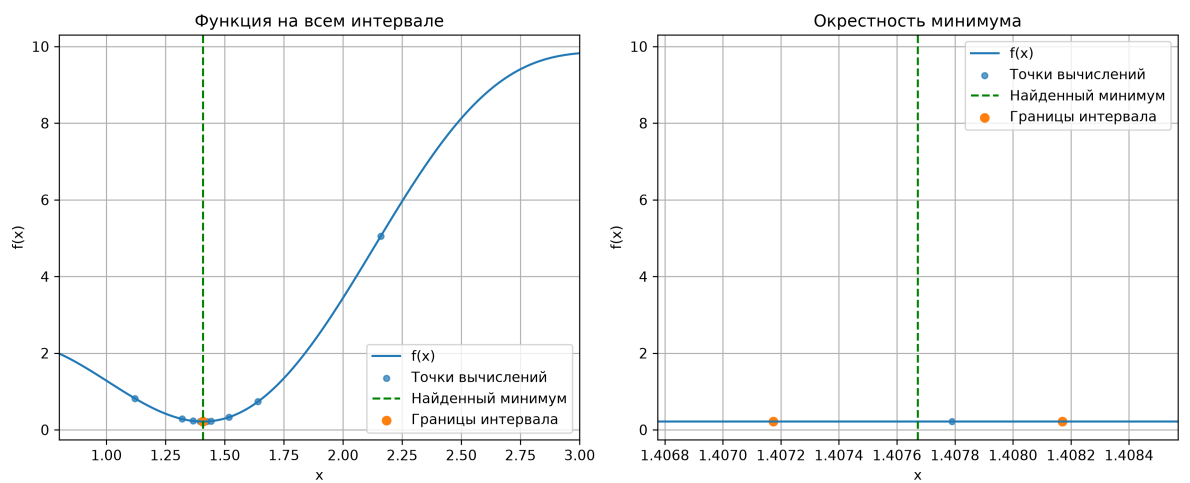


Рис. 9. Метод золотого сечения на функции f_2 .

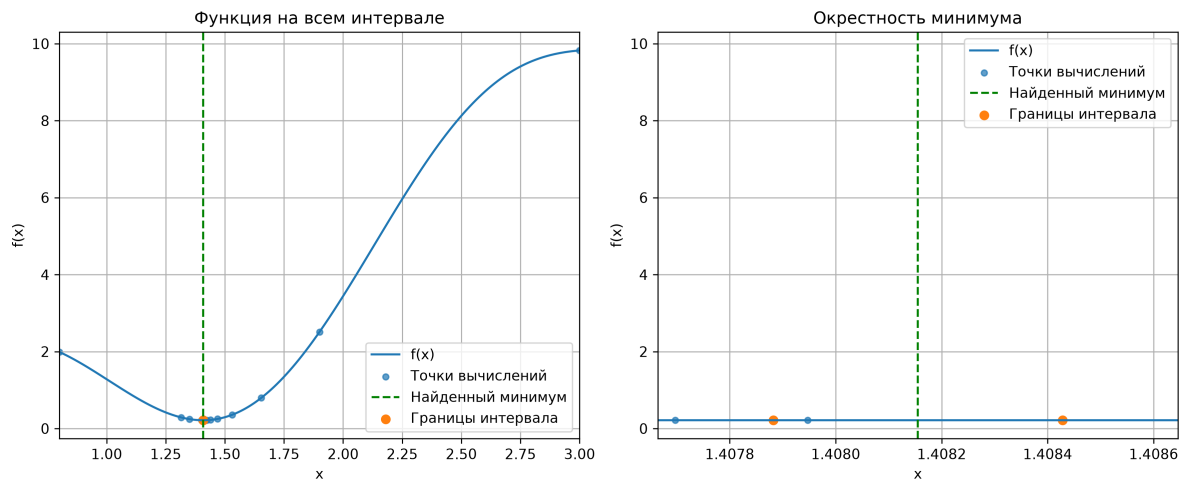


Рис. 10. Метод парабол на функции f_2 .

Функция f_2 имеет более сложный рельеф из-за синусоидальной составляющей. Тем не менее на выбранном интервале она сохраняет унимодальное поведение, поэтому локальные методы сходятся к одному и тому же минимуму. Полученные графики подтверждают, что уменьшение числа вычислений достигается за счет повторного использования уже вычисленных внутренних точек.

4.2. Скорость сходимости локальных методов

На рисунках 11 и 12 по вертикальной оси отложена длина текущего отрезка локализации в логарифмическом масштабе, а по горизонтальной — число вычислений функции. Такое представление позволяет сравнивать методы с разными законами сходимости на одной диаграмме.

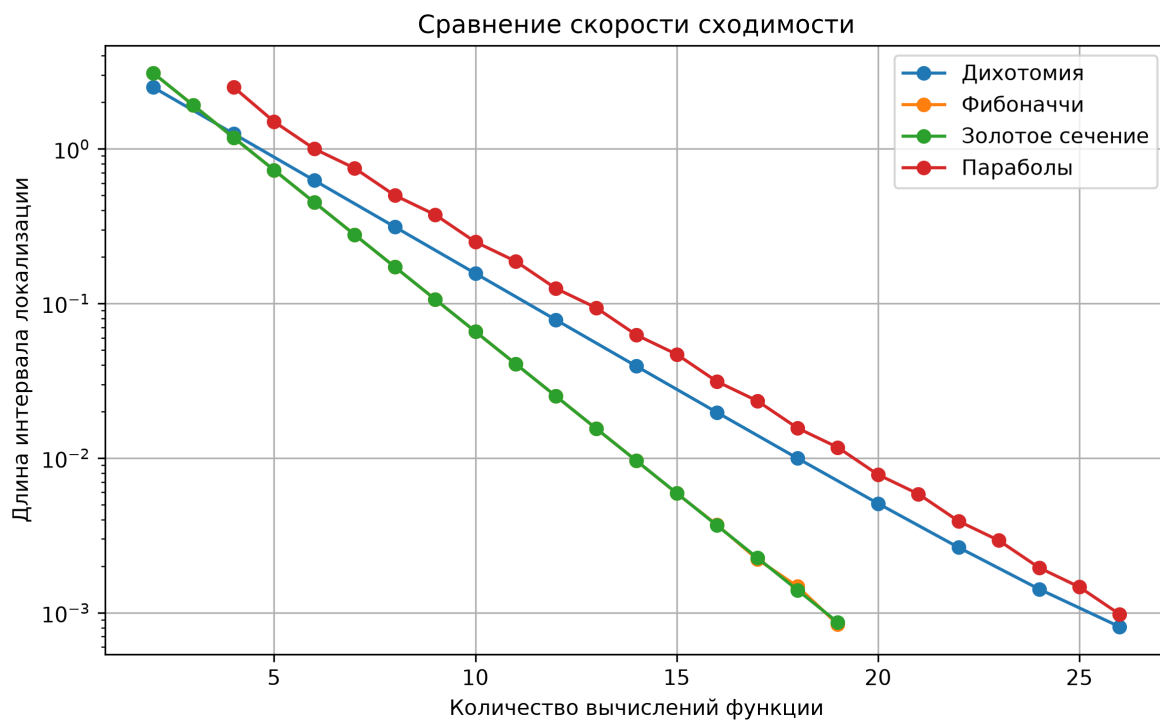


Рис. 11. Скорость сходимости локальных методов на функции f_1 .

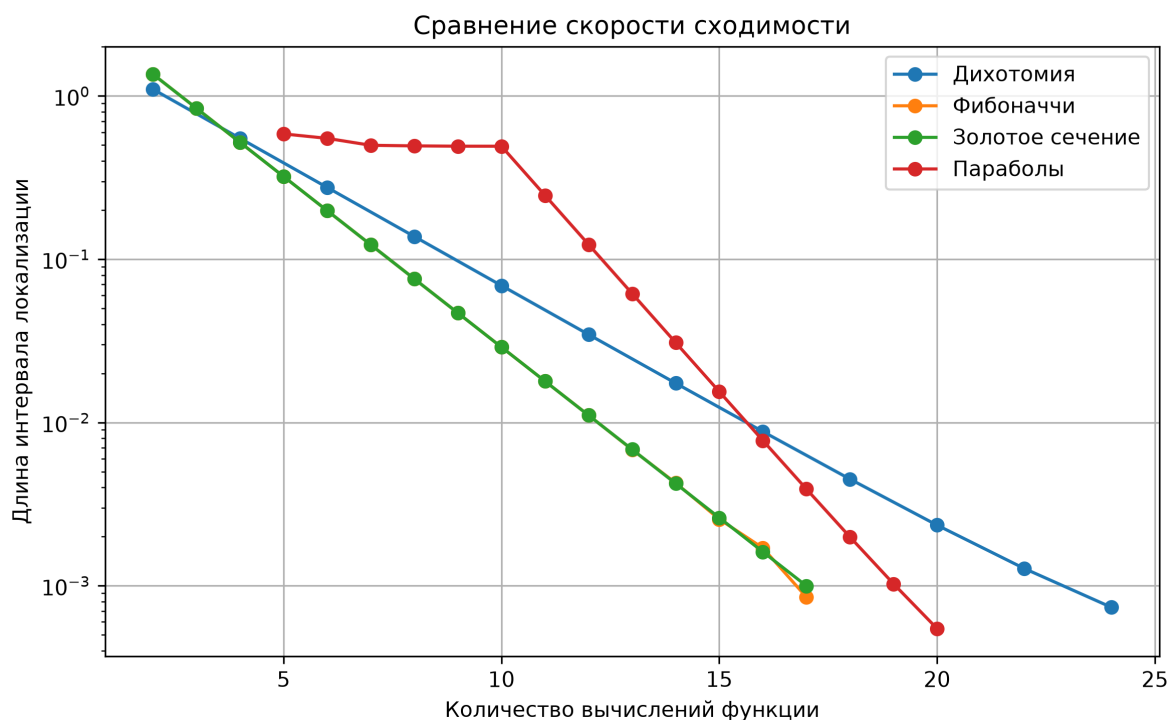


Рис. 12. Скорость сходимости локальных методов на функции f_2 .

По графикам видно, что методы Фибоначчи и золотого сечения обеспечивают устойчивое уменьшение длины интервала при малом числе вычислений. Метод парабол может сходиться быстрее на гладких участках функции, однако его эффективность зависит от качества квадратичной аппроксимации. Пассивный поиск не строит итерационную последовательность локализации и поэтому на графике сходимости не представлен.

4.3. Глобальные методы

Для мультимодальной функции f_3 применялись равномерный перебор и метод ломаных. Результаты равномерного перебора и метода ломаных показаны на рисунках 13 и 14 соответственно.

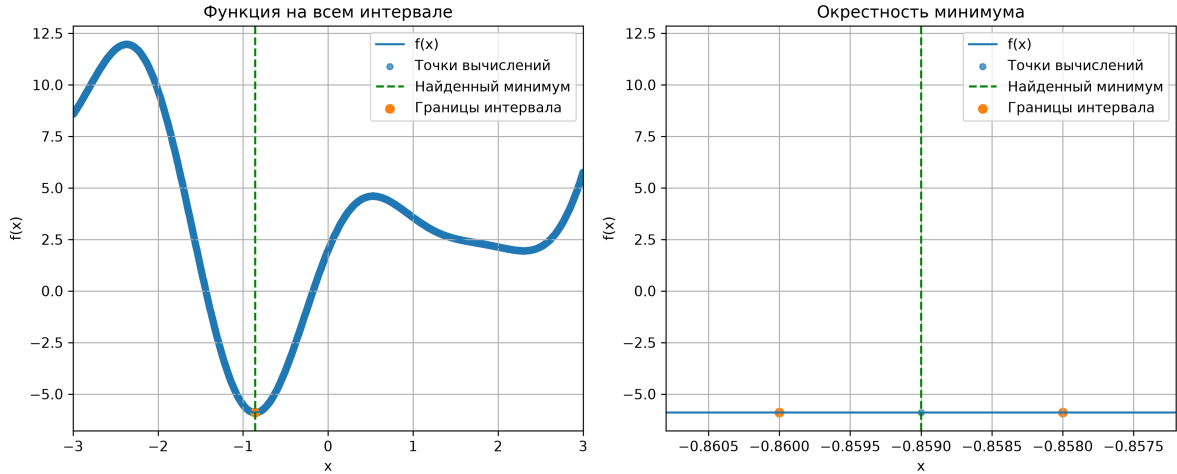


Рис. 13. Равномерный перебор на функции f_3 .

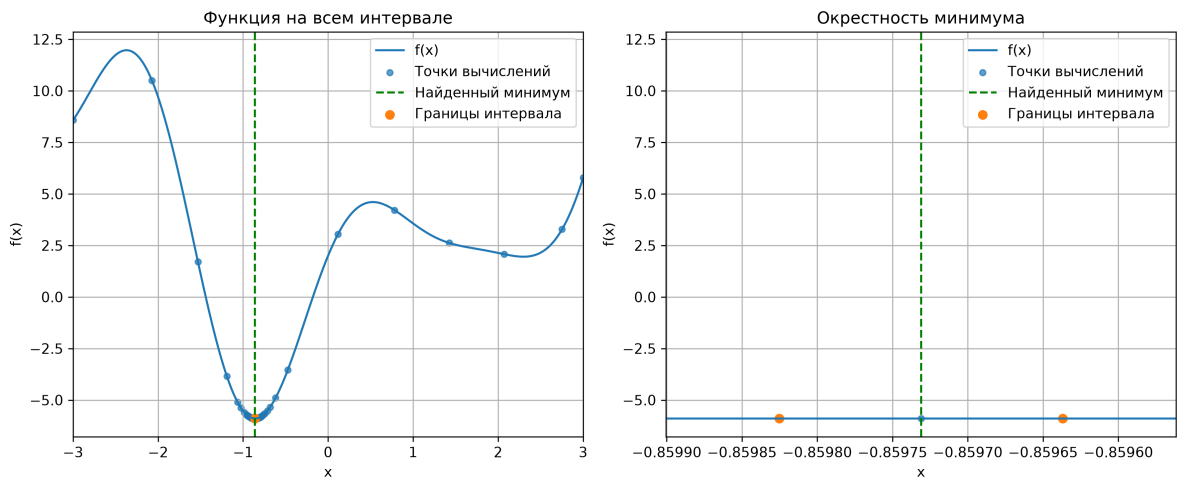


Рис. 14. Метод ломаных на функции f_3 .

Равномерный перебор исследует весь интервал с одинаковым шагом, поэтому его поведение не зависит от локальной структуры функции. Метод ломаных распределяет новые вычисления адаптивно: больше точек появляется в интервалах, которые имеют наиболее перспективную нижнюю оценку. Это позволяет использовать информацию о константе Липшица и уточнять область глобального минимума.

4.4. Сравнение при одинаковом числе вычислений

Для корректного сравнения глобальных методов им был задан одинаковый бюджет вычислений функции. На рисунках 15 и 16 показаны ошибки координаты минимума и значения функции относительно эталонного решения, полученного на плотной сетке.

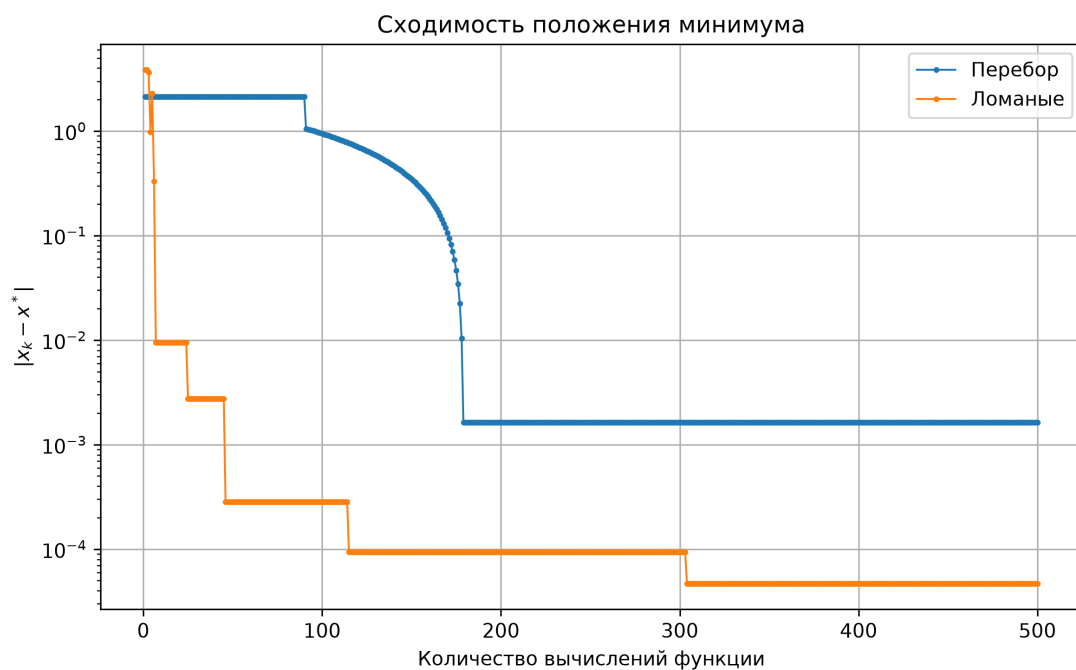


Рис. 15. Ошибка координаты минимума при одинаковом числе вычислений.

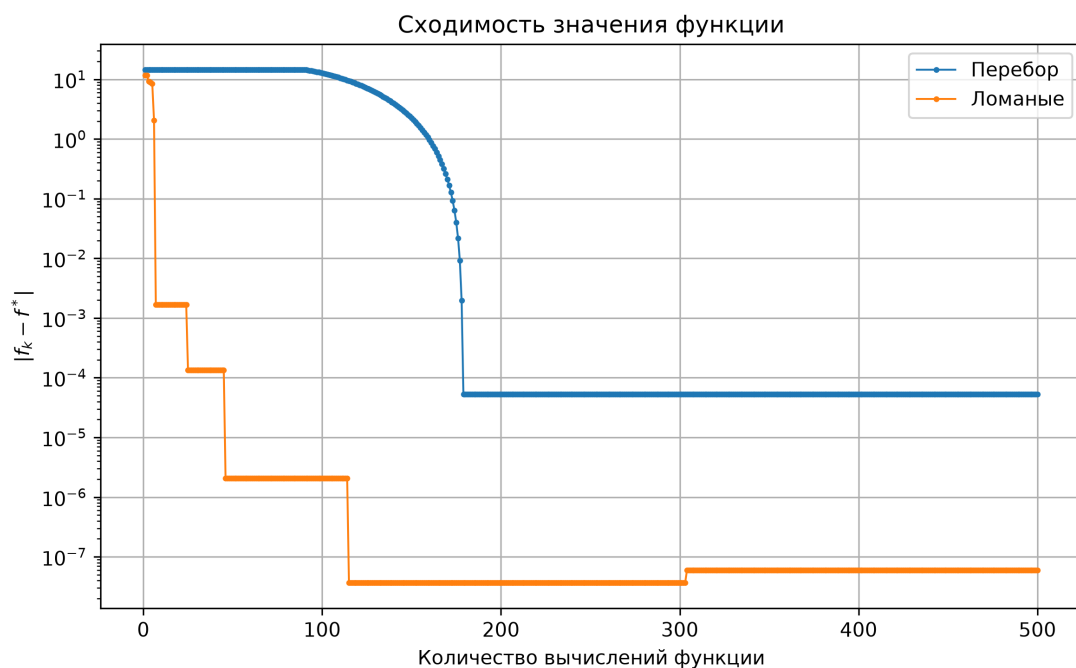


Рис. 16. Ошибка значения функции при одинаковом числе вычислений.

Эти графики показывают, что метод ломаных обеспечивает более точное приближение глобального минимума по сравнению с равномерным перебором при одинаковом числе вычислений функции. Метод перебора требует значительно большего числа вычислений для достижения сопоставимой точности и быстро выходит на плато из-за ограниченной ширины равномерной сетки.

Отдельно выполнено сравнение локальных методов при одинаковом числе вычислений. Результат представлен на рисунках 17 и 18.

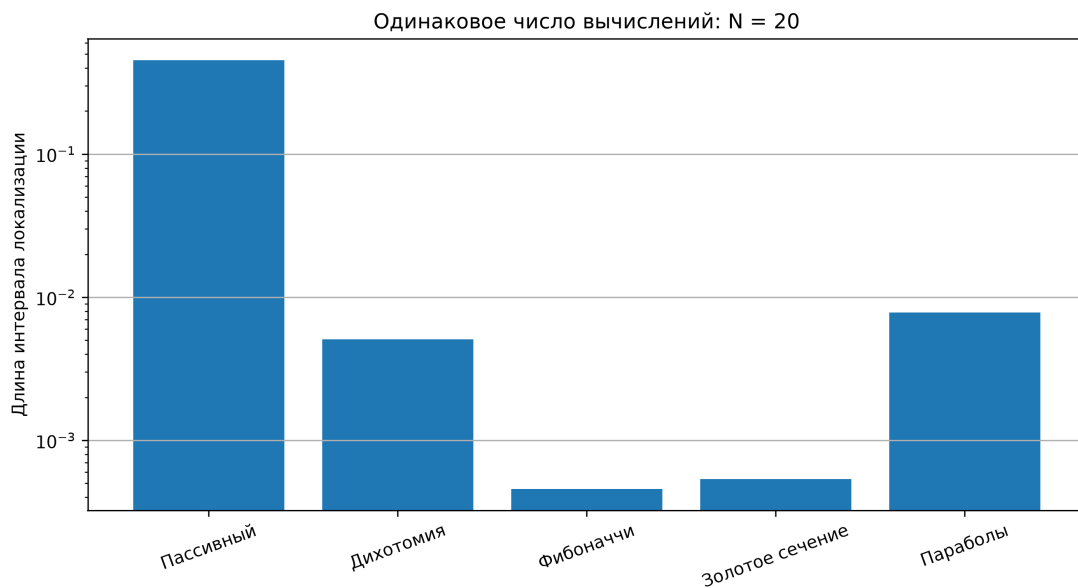


Рис. 17. Сравнение длины итогового интервала при одинаковом числе вычислений для f_1 .

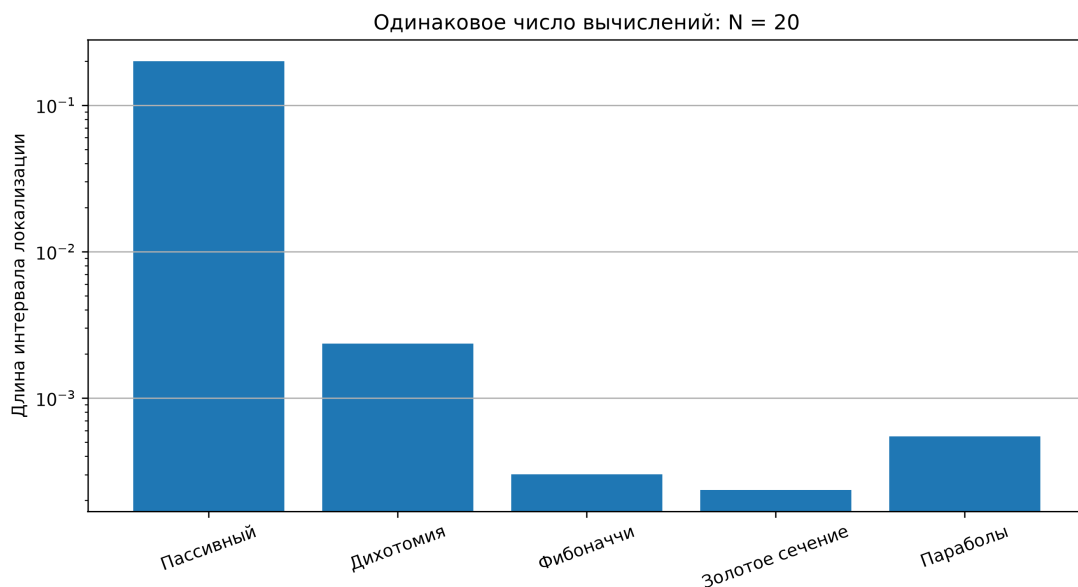


Рис. 18. Сравнение длины итогового интервала при одинаковом числе вычислений для f_2 .

5. Вывод

В работе исследованы локальные и глобальные методы одномерной минимизации. Эффективность методов оценивалась по длине итогового отрезка локализации, числу вычислений функции и скорости уменьшения ошибки.

Основные результаты:

1. **Метод Фибоначчи показал наилучшую эффективность при фиксированном числе вычислений.** Он оптимально использует доступный вычислительный бюджет, поскольку на каждой итерации сохраняет одно из ранее вычисленных значений функции.
2. **Метод золотого сечения практически сопоставим с методом Фибоначчи.** Его преимущество заключается в более простой реализации и возможности остановки на любой итерации без предварительного выбора конечного числа шагов.
3. **Метод дихотомии является надежным, но менее экономичным.** На каждой итерации он требует двух новых вычислений функции, поэтому при одинаковом вычислительном бюджете обычно уступает методам Фибоначчи и золотого сечения.
4. **Пассивный поиск является самым простым, но наименее эффективным локальным методом.** Для повышения точности ему приходится равномерно исследовать весь интервал, что приводит к большому числу вычислений функции.
5. **Метод парабол эффективен для гладких функций.** Использование квадратичной аппроксимации позволяет быстро определить положение минимума. Однако метод чувствителен к выбору трех точек и может работать нестабильно, если построенная парабола вырождается или ее вершина выходит за пределы текущего интервала.
6. **Локальные методы нельзя напрямую применять к мультимодальным функциям.** В этом случае они могут сойтись к одному из локальных минимумов, не обнаружив глобальный минимум.
7. **Для мультимодальной функции метод ломаных является более содержательным, чем равномерный перебор.** Он распределяет вычисления адаптивно и использует оценку Липшица, поэтому способен уделять больше внимания наиболее перспективным участкам интервала.
8. **Равномерный перебор отличается предсказуемостью, но требует большого числа вычислений.** Его преимущество состоит в простоте и равномерном исследовании всего интервала, однако он не использует информацию о форме функции.
9. **Выбор метода должен зависеть от структуры целевой функции.** Для унимодальных функций наиболее рациональны методы Фибоначчи и золотого сечения, а для

гладких функций с устойчивой аппроксимацией дополнительно эффективен метод парабол. Для мультимодальных функций следует применять методы глобального поиска.

Таким образом, наиболее универсальным выбором среди рассмотренных локальных методов является метод золотого сечения: он близок по эффективности к методу Фибоначчи, но проще в реализации и допускает остановку в произвольный момент. Метод Фибоначчи предпочтителен, когда заранее задано точное число вычислений. Метод парабол способен дать лучший результат на гладких функциях, но требует дополнительного контроля устойчивости. Для функций с несколькими экстремумами следует отдавать предпочтение методу ломаных, который адаптивно распределяет вычисления и использует информацию о форме функции, что позволяет более эффективно находить глобальный минимум.

Литература

- [1] Сухарев А. Г., Тимохов А. В., Федоров В. В. *Курс методов оптимизации*. — Москва: Наука, 1986.